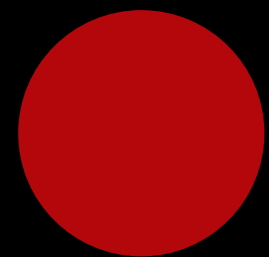
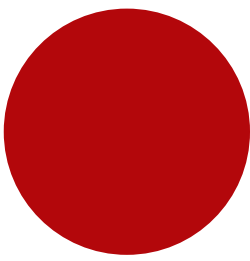


FACULTAD DE INGENIERÍA Y ARQUITECTURA
INGENIERÍA EN SISTEMAS
ALGORITMOS Y ESTRUCTURAS DE DATOS

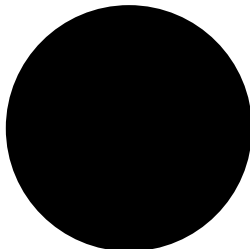
PROYECTO DE INVESTIGACIÓN ANÁLISIS RADIX SORT Y BÚSQUEDA BINARIA

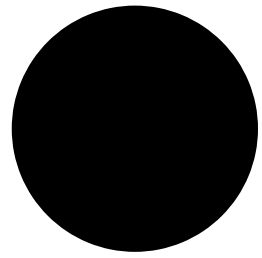


Genesis Nicole Ramos Zambrana
Allysson Gabriela Palma Alvarado
Andres Humberto Porras Romero



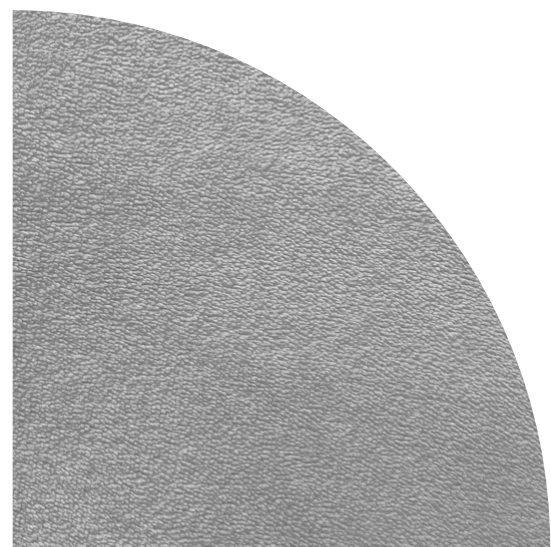
CONTENIDO

- Introducción
 - Planteamiento del problema
 - Objetivo de la investigación
 - Objetivos específicos
 - Metodología
 - Marco conceptual / referencial
 - Implementación del algoritmo
 - Análisis a Priori
 - Análisis a Posteriori
 - Resultados
 - Conclusiones
 - Anexos
 - Referencias bibliográficas
- 
- 



INTRODUCCIÓN

La presente investigación tiene como propósito analizar y comparar la eficiencia de dos algoritmos fundamentales en el ámbito de la informática: Radix Sort y Búsqueda Binaria. Estos algoritmos juegan un papel crucial en la gestión y procesamiento de grandes volúmenes de datos, ya que permiten organizar y acceder a la información de manera eficiente.





Planteamiento del problema



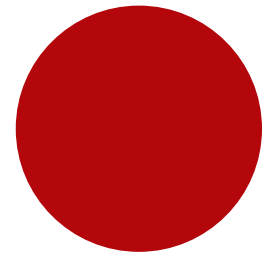
LA EFICIENCIA DE LOS ALGORITMOS DE ORDENAMIENTO Y BÚSQUEDA ES FUNDAMENTAL EN EL DESARROLLO DE APLICACIONES QUE MANEJAN GRANDES VOLÚMENES DE DATOS. LA ELECCIÓN DEL ALGORITMO ADECUADO PUEDE REDUCIR SIGNIFICATIVAMENTE EL TIEMPO DE PROCESAMIENTO Y EL USO DE RECURSOS COMPUTACIONALES. SIN EMBARGO, NO TODOS LOS ALGORITMOS RINDEN IGUAL EN DIFERENTES CONTEXTOS; ALGUNOS PUEDEN SER MÁS RÁPIDOS PERO MÁS EXIGENTES EN MEMORIA, MIENTRAS QUE OTROS PUEDEN SER MÁS EFICIENTES PARA CIERTOS TIPOS DE DATOS.

Objetivo de la investigación

Analizar y comparar la eficiencia de algoritmos de ordenamiento y búsqueda ejecutados en distintos dispositivos, evaluando su desempeño en un sistema de impresión de notas.

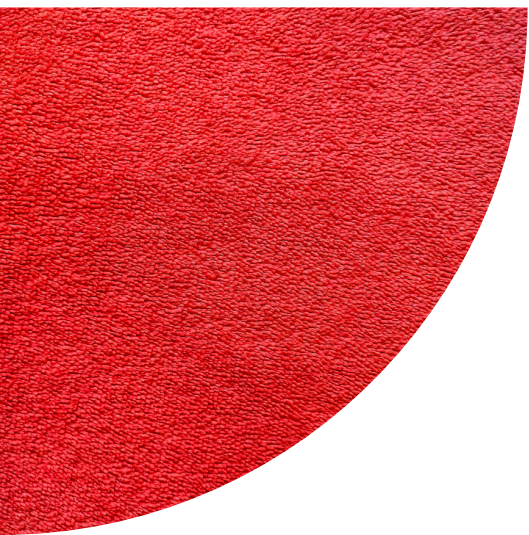
Objetivos específicos

- Analizar la eficiencia de los algoritmos radix sort y búsqueda binaria en relación al tiempo de ejecución y consumo de memoria
- Implementar un sistema de impresión de notas en Python utilizando el algoritmo Radix Sort el cual ordene los datos de los estudiantes incluyendo (Cif, edad y calificación), Aplicando el algoritmo de búsqueda binaria dentro del sistema ordenado para localizar eficientemente la nota de un estudiante específico, evaluando su rapidez y precisión.
- Comparar el rendimiento de los algoritmos diferentes entornos de ejecución implementados en función del tiempo de ejecución y uso de memoria considerando el tipo de dispositivo



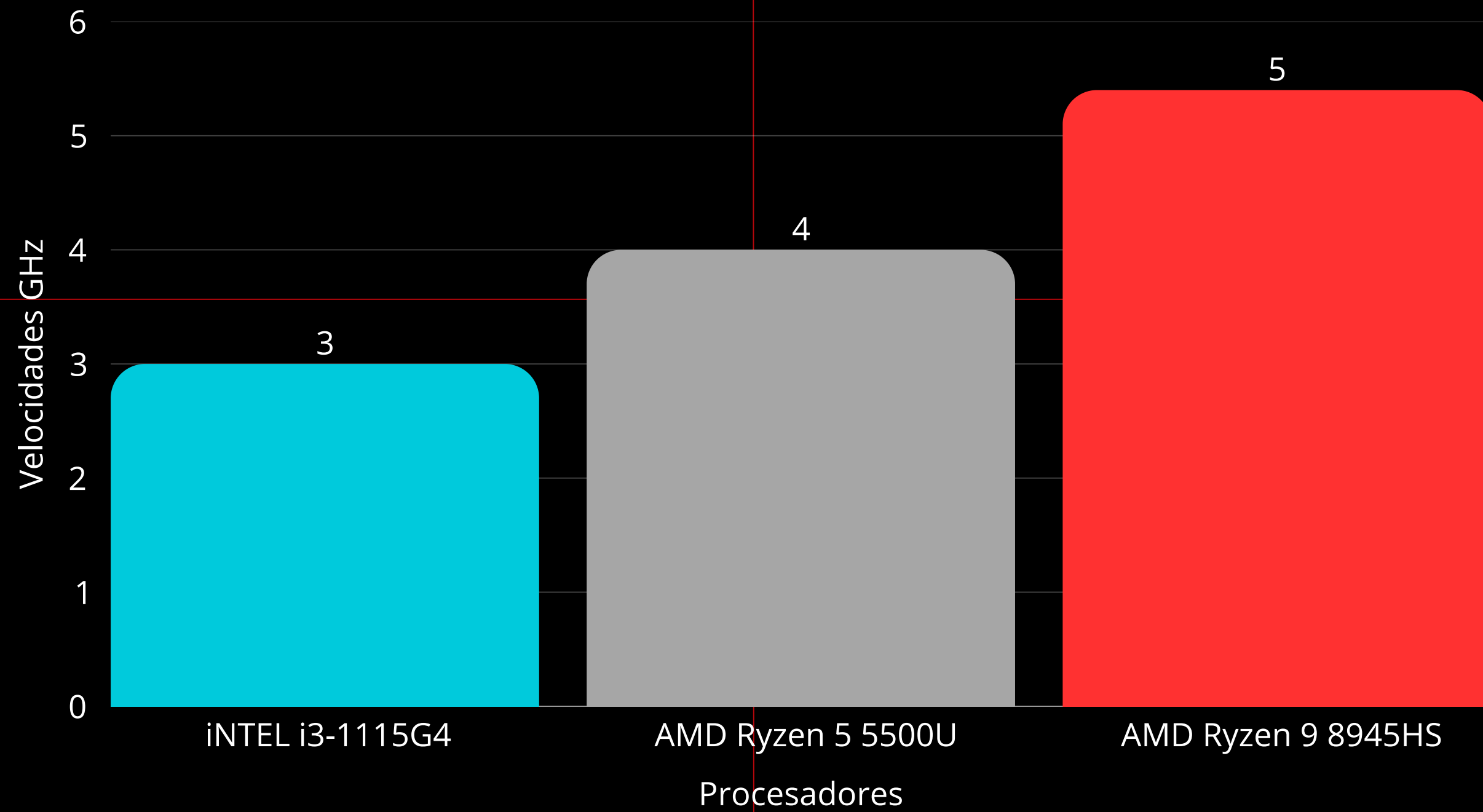
METODOLOGÍA

1. Diseño de la investigación
2. Enfoque de la investigación
3. Alcance de la investigación
4. Procedimiento



SIMULACIÓN GRÁFICA

Velocidades de procesadores GHz

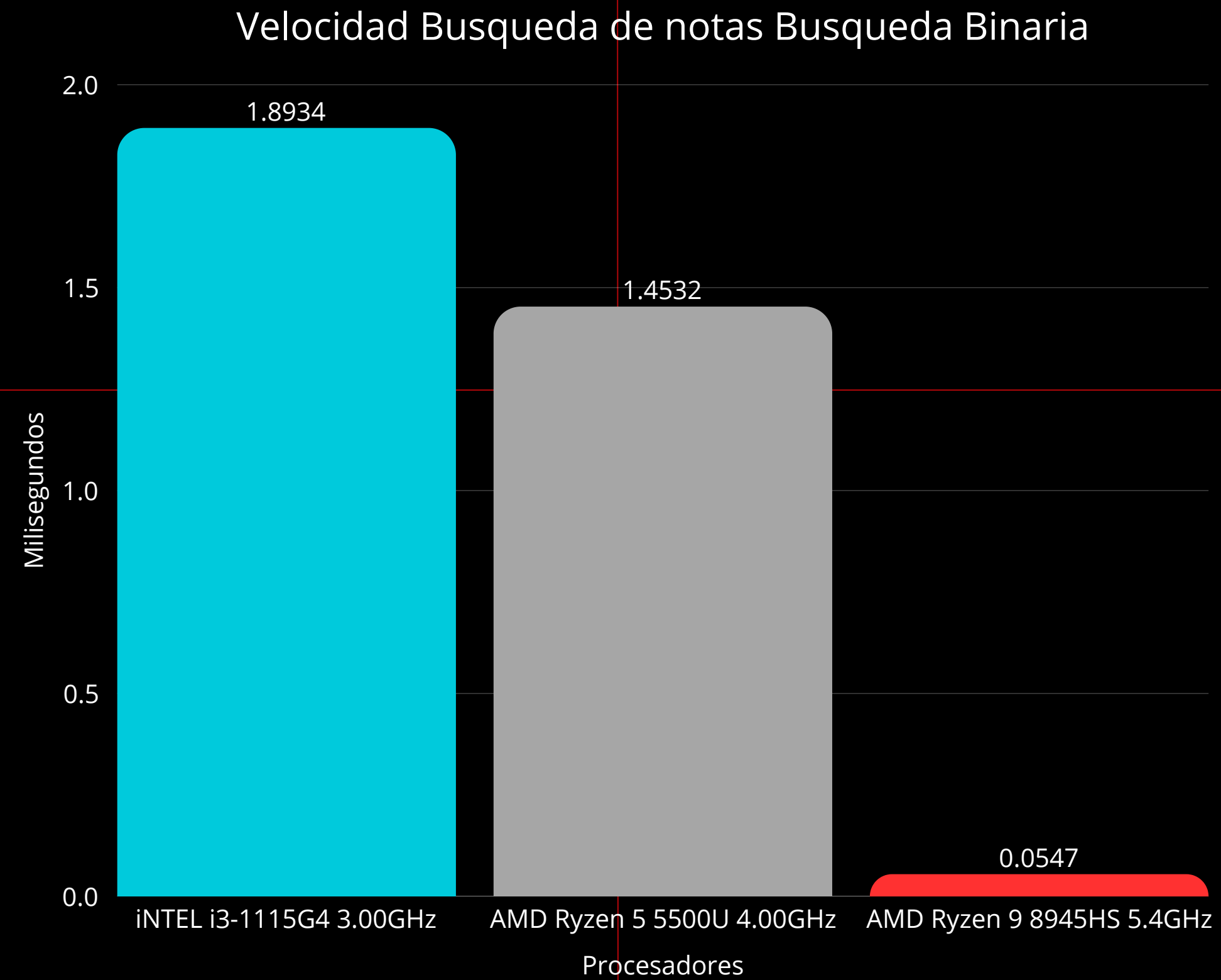


SIMULACIÓN GRÁFICA RADIX SORT

Velocidad Ordenamiento de calificaciones Radix Sort

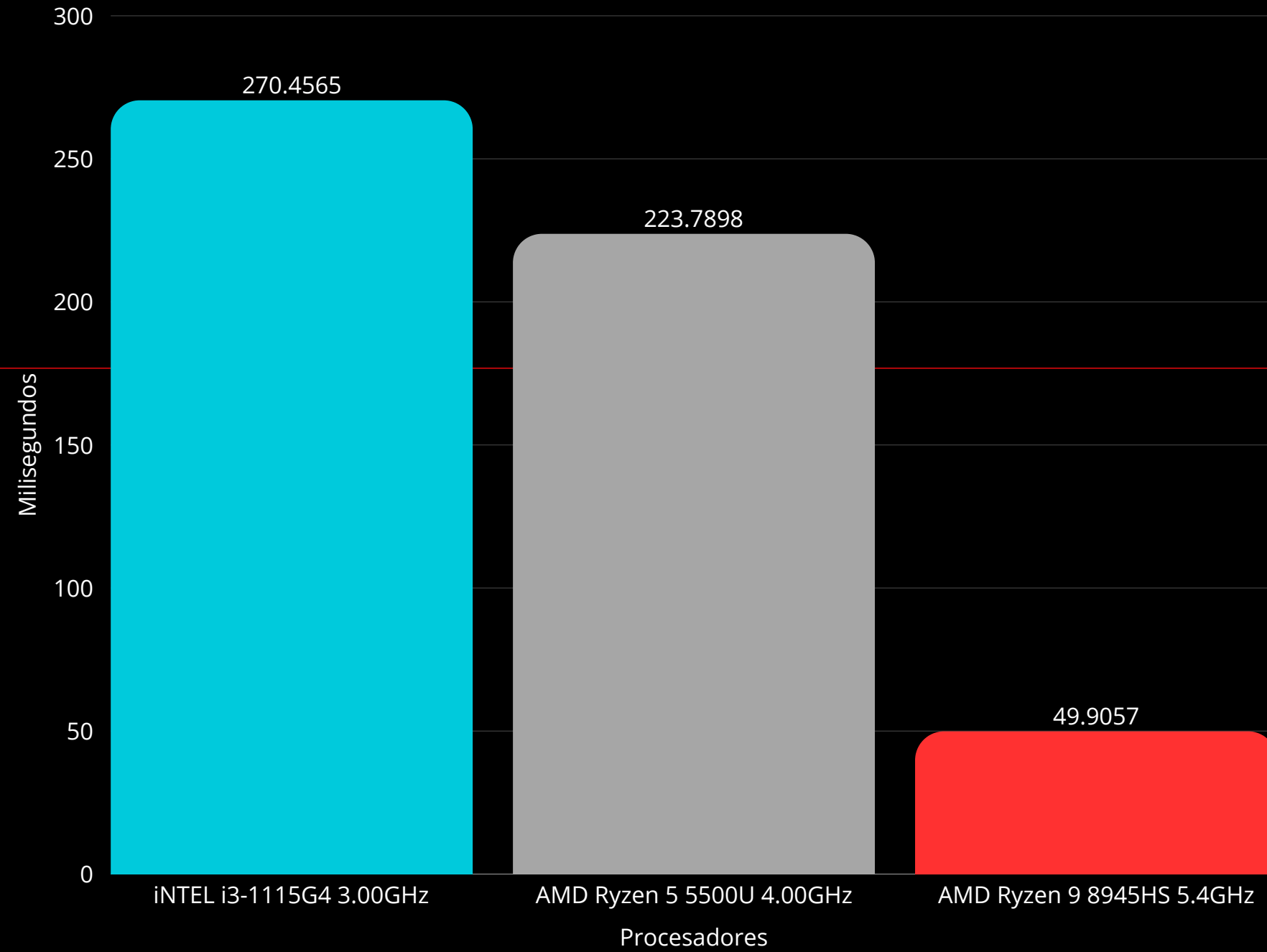


SIMULACIÓN GRÁFICA BUSQUEDA BINARIA

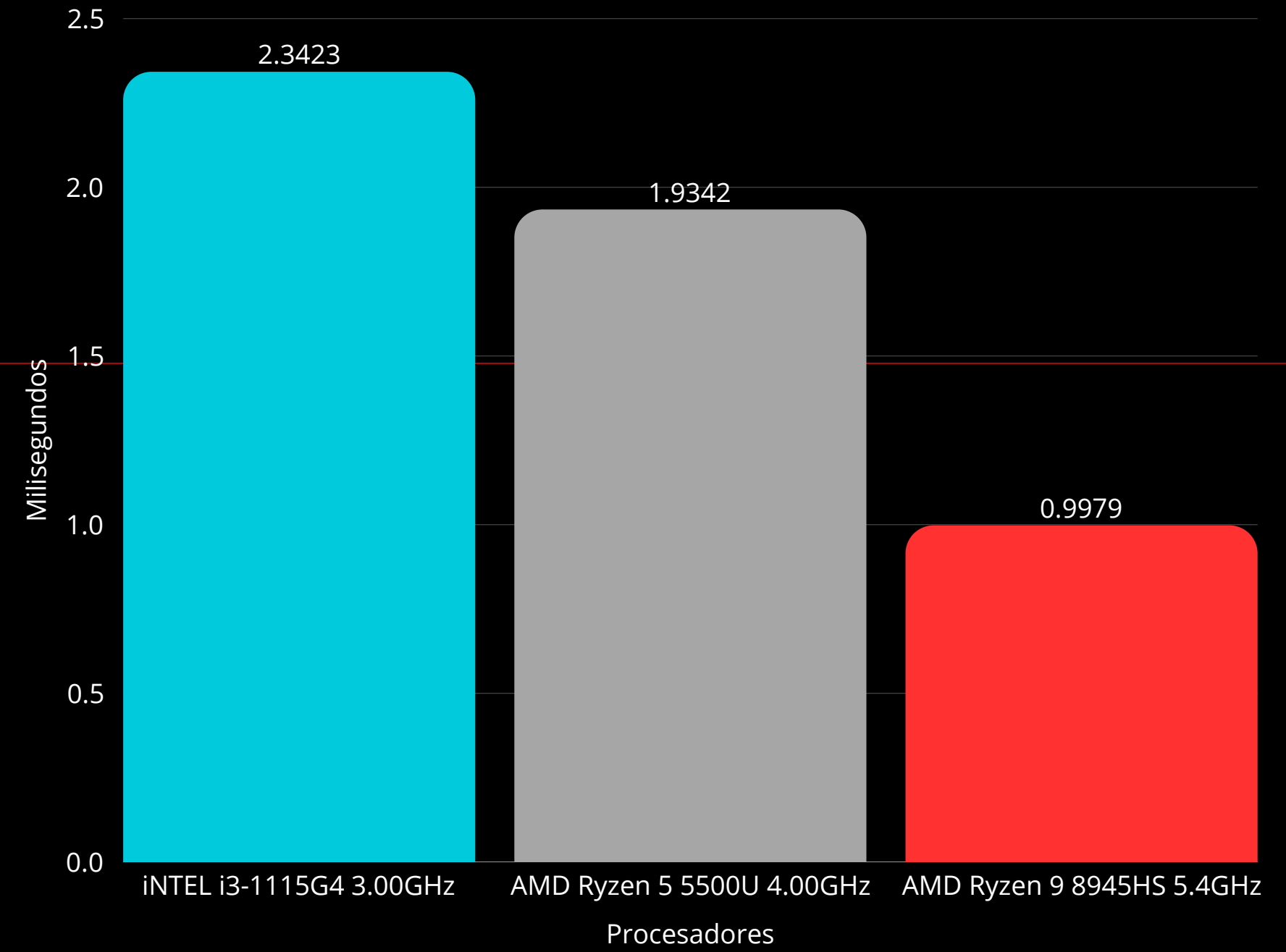


BUSQUEDA LINEAL Y MERGE SORT

Velocidad Ordenamiento de calificaciones Merge Sort

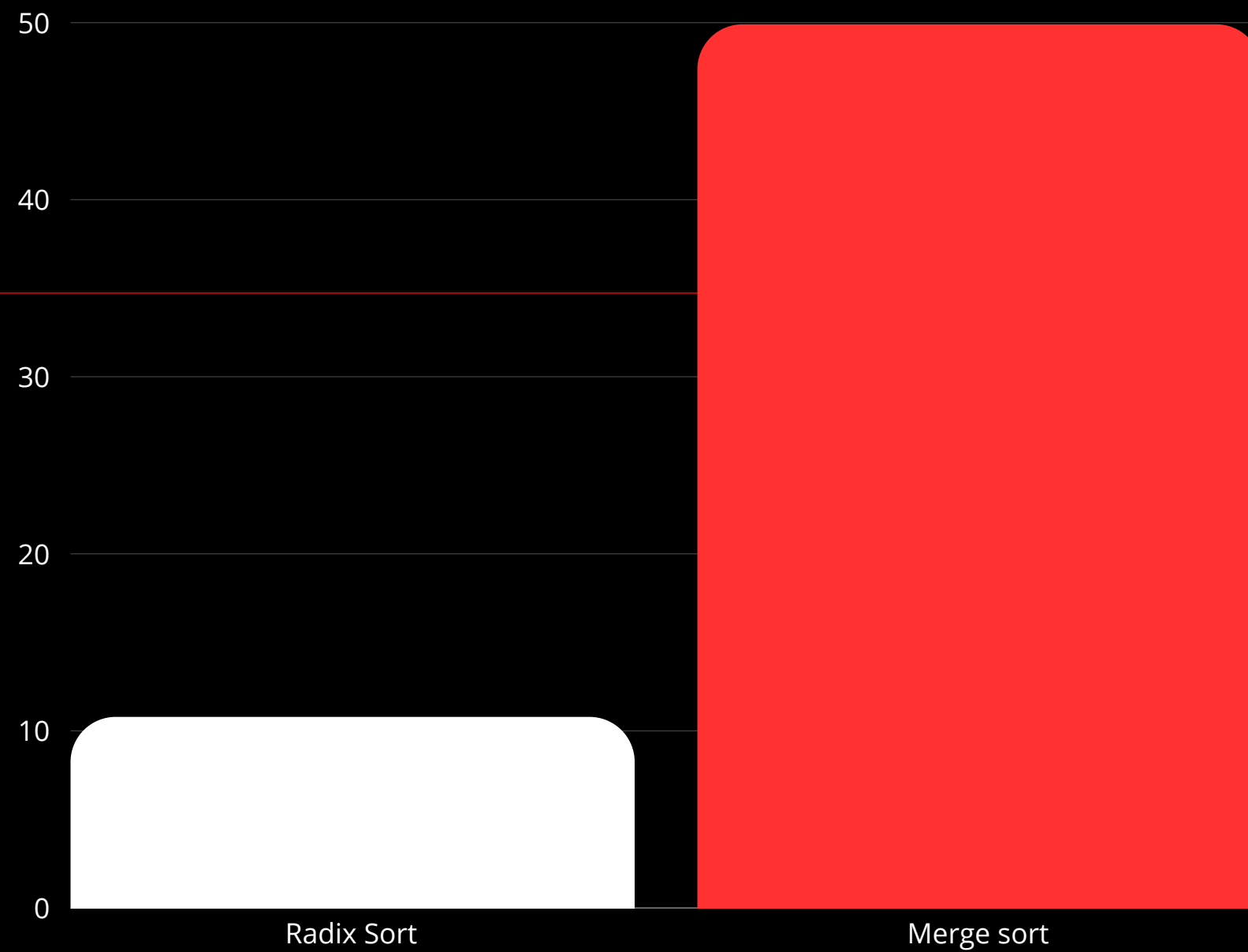


Velocidad Busqueda de notas Busqueda Lineal

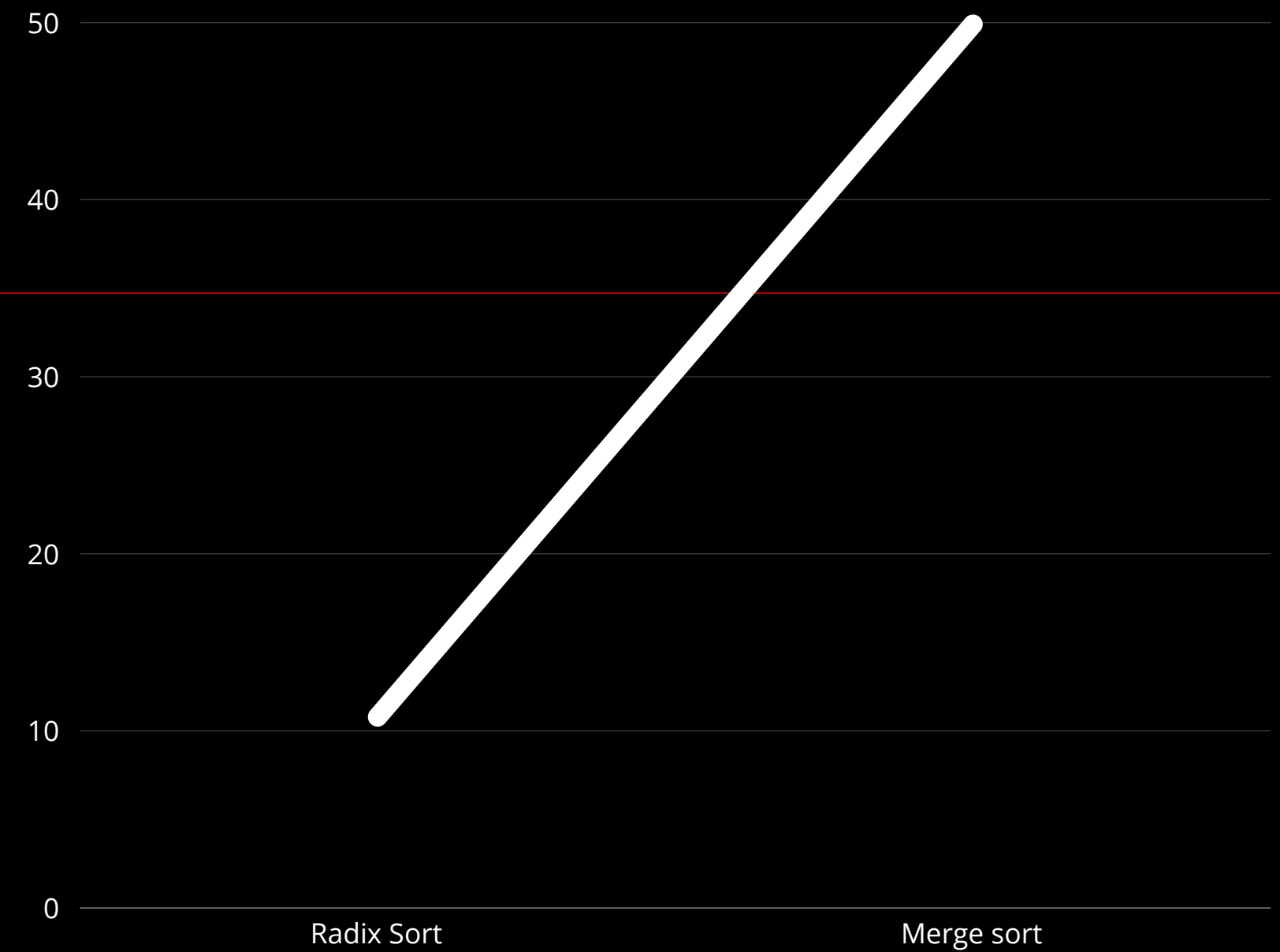


RADIX SORT VS MERGE SORT

Radix Sort Ordenamiento VS Merge Sort Ordenamiento
Procesador AMD Ryzen 9 8945HS

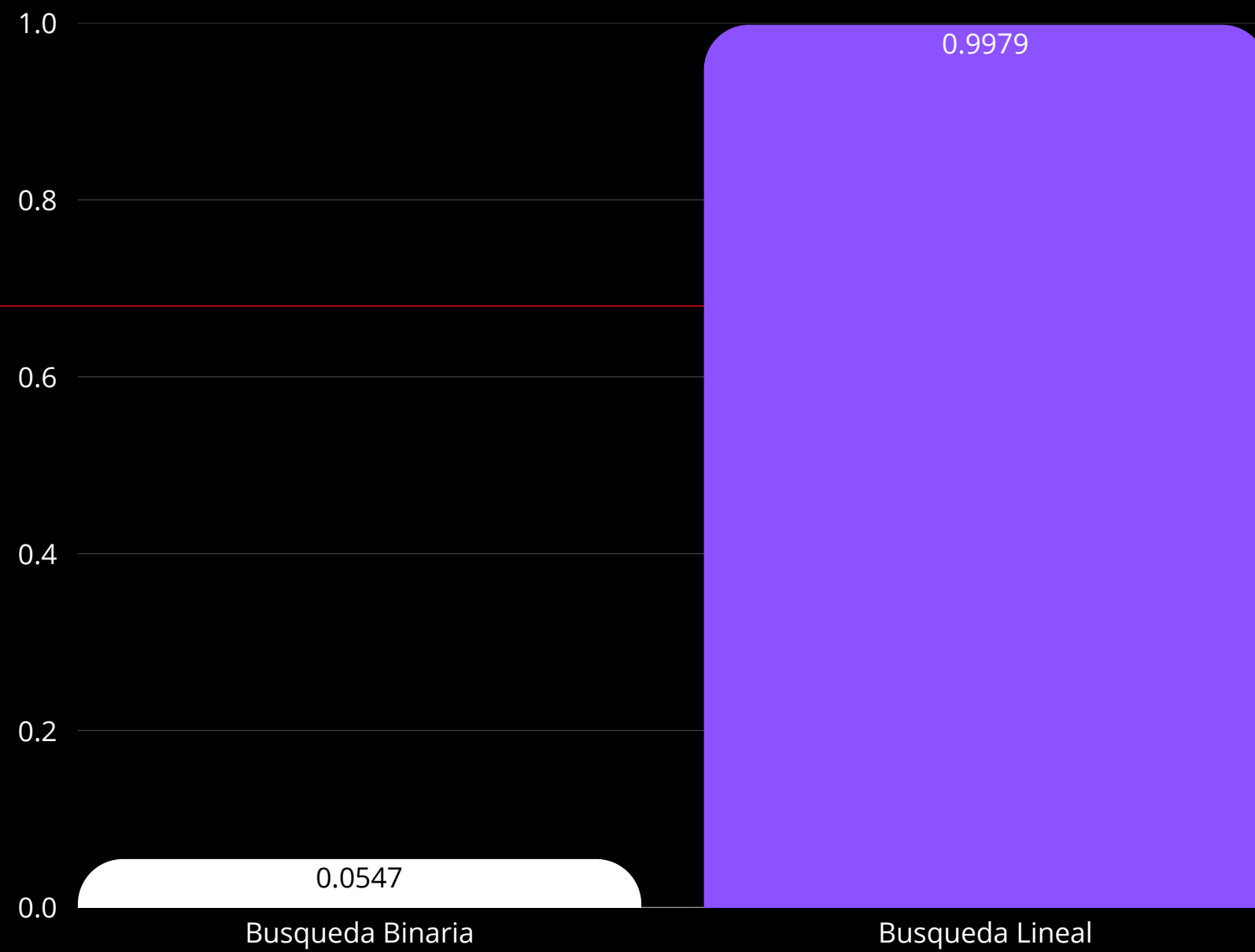


Radix Sort VS Merge Sort
Procesador AMD Ryzen 9 8945HS

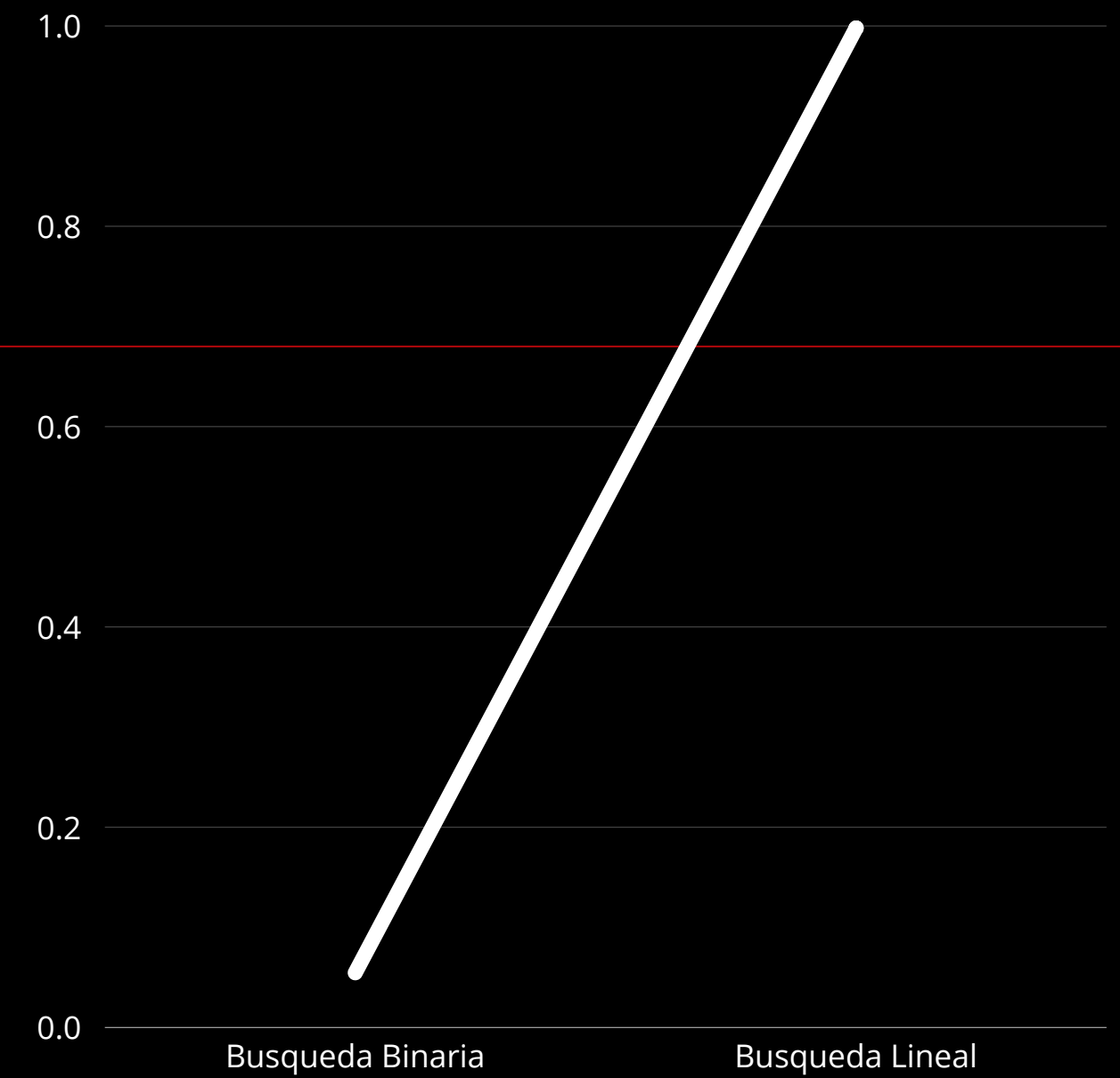


BUSQUEDA BINARIA VS BUSQUEDA LINEAL

Busqueda Binaria VS Busqueda Lineal
Procesador AMD Ryzen 9 8945HS



Busqueda Binaria VS Busqueda Lineal
Procesador AMD Ryzen 9 8945HS



¿Por qué la Búsqueda Binaria es tan superior?

1. Búsqueda Lineal: El método "Fuerza Bruta"

La Búsqueda Lineal es el método más simple. Funciona como buscar un libro en un estante desordenado: empiezas por el primero y revisas cada uno, uno por uno, hasta que encuentras el que buscas.

2. Búsqueda Binaria: El método "Divide y Vencerás"

La Búsqueda Binaria es mucho más inteligente, pero necesita una condición fundamental: la lista debe estar ordenada. Este sistema ya cumple esto al usar primero un método de ordenamiento.

¿Por qué Radix Sort es más rápido aquí?

1. No hace comparaciones:

Radix Sort no compara un número con otro. Simplemente distribuye los elementos en "cubetas" basándose en sus dígitos. Este proceso es mecánicamente muy rápido.

2. Complejidad Lineal para tus Datos:

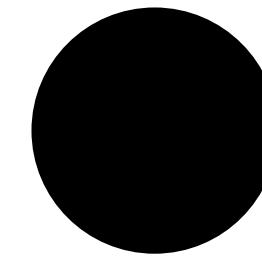
La complejidad de Radix Sort es $O(d*n)$, donde d es el número de dígitos de tus números y n es la cantidad de datos. Tus calificaciones (máximo 100) y edades (máximo 25) tienen solo 2 o 3 dígitos. Esto hace que d sea un número muy pequeño y constante.

IMPLEMENTACIÓN DEL ALGORITMO

ANÁLISIS A PRIORI

En análisis a priori en cuanto se refiere a eficiencia espacial tenemos que radix sort no es un algoritmo de ordenamiento in place lo cual significa que requiere una memoria adicional para completar su ejecución, en cuanto a búsqueda binaria es un algoritmo que tiene una eficiencia muy alta por lo que puede implementarse de manera interactiva en eficiencia temporal radix sort la eficiencia depende de la longitud máxima de números en dígitos. En la búsqueda binaria dado a que esta se aplica sobre una lista previamente ordenada su eficiencia es $O(\log n)$ ya que cada iteración descarta la mitad del espacio de búsqueda. En el Análisis de Orden Radix sort si se considera que d es un dato constante ya sea por las notas donde n es el número de elementos y d son los dígitos del número $O(d*n)$, o por el cif es $O(n)$ ya que en este caso no crece con n por que se considera un dato fijo.

En el caso de búsqueda binaria es $O(\log n)$ esto significa que incluso para una lista muy grande el número de comparación sigue siendo bajo es por eso que su notación en big o es de esta forma.



ANÁLISIS A POSTERIORI

En el mejor de los casos los datos requieren un procesamiento mínimo. Sin embargo radix sort no mejora con el orden inicial de los datos ya que no compara elementos directamente sino que organiza por dígitos desde la posición menos significativa hasta la más significativa. En cuanto a búsqueda binaria, el mejor caso ocurre cuando la calificación buscada está en el centro de la lista ordenada. En el caso promedio ocurre cuando los datos numéricos de longitud moderada y distribuidos aleatoriamente como es el caso de las calificaciones generadas entre 40 y 100, radix sort mantiene un comportamiento eficiente y constante, su rendimiento no se ve afectado por el orden si no por la estructura interna de los valores, la búsqueda binaria sobre la lista ordenada por calificación permite encontrar cualquier valor con pocas comparaciones, en el peor de los casos para radix sort no se debe al orden inverso, como sucede con algoritmos como bubble sort, sino a la longitud de los valores numéricos. Si el dato clave como el cif tiene muchos dígitos aumenta la cantidad de pasadas necesarias y en búsqueda binaria sería que el dato de la calificación no exista en la lista.

CONCLUSIONES

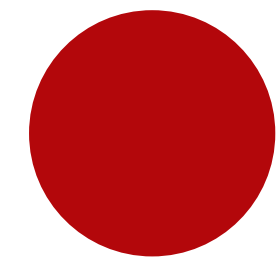
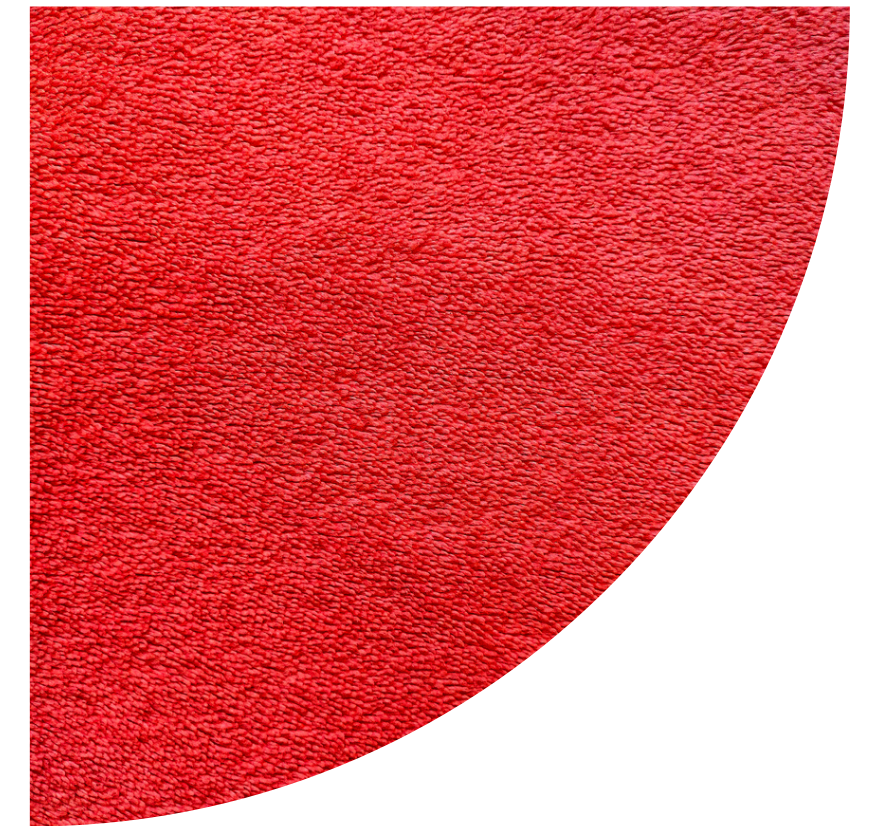
En base a los análisis realizados y la implementación práctica del sistema realizado, evaluando así el comportamiento de ambos algoritmos aplicados en dos escenarios los cuales fueron ordenamiento por calificación y edad llegamos a la conclusión de que radix sort demostró ser un algoritmo eficiente para ordenar datos numéricos como calificaciones y edades, manteniendo un rendimiento constante en todos los casos gracias a su complejidad lineal $O(n \cdot d)$.

La búsqueda binaria aplicada sobre la lista ordenada por calificación permitió localizar valores de forma rápida y con bajo consumo de recursos destacando por su eficiencia logarítmica de $O(\log n)$.

Aunque el sistema también emplea búsqueda lineal por cif, esta técnica es menos eficiente, sin embargo resulta útil en contextos donde no se requiere ordenamiento previo.

Las mediciones realizadas en tiempo de ejecución y uso de memoria muestran que ambos algoritmos se adaptan bien a bases de datos medianas como los 5000 registros utilizados en el sistema.

GRACIAS POR SU ATENCIÓN



LOGO